

CSE 331: Computer Security

Fundamentals

Spring 2026

Exploit Chain &
Return-Oriented Programming

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

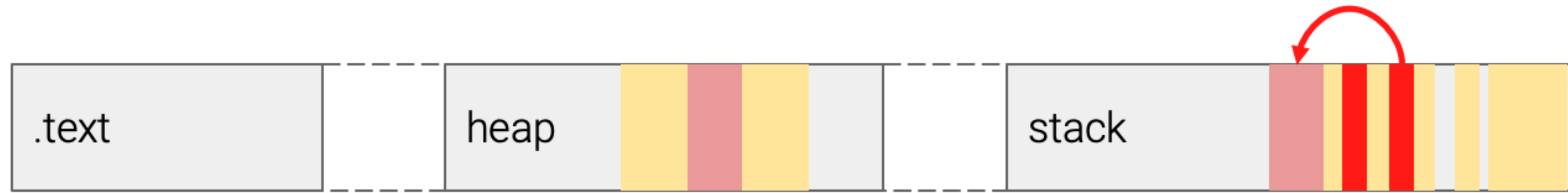
安徽大学纽约石溪学院

Stages of an Attack

1. External reconnaissance: Understanding the program, how it works, where are the potential vulnerabilities.
2. Gaining a foothold: Find the first exploit, even if small.
3. Internal reconnaissance: What does it change? What else can you do now?
4. Gaining influence: Exploit new vulnerabilities opened up.
5. Total compromise: Go to 3 until total compromise.
6. Profit: \$\$\$

Control Flow Hijack to shellcode

Mixing control information and data



- Memory corruption occurs when user-controlled data manages to spread into data that shouldn't be user controlled (through a memory error).
- If you overwrite control data (i.e., a return address), you can use this to redirect control flow elsewhere.
- Or you can also redirect it to your injected code.

Shellcode example

```
void bye1() { puts("Goodbye!"); }
void bye2() { puts("Farewell!"); }
void hello(char *name, void (*bye_func)())
{
    printf("Hello %s!\n", name);
    bye_func();
}
int main(int argc, char **argv)
{
    char name[1024];
    gets(name);
    srand(time(0));
    if (rand() % 2) hello(bye1, name);
    else hello(name, bye2);
}
```

Compile with: gcc -fno-stack-protector -z execstack -o hello hello.c

Run with: setarch x86_64 -R ./hello

Shellcode example

```
void bye1() { puts("Goodbye!"); }
void bye2() { puts("Farewell!"); }
void hello(char *name, void (*bye_func)())
{
    printf("Hello %s!\n", name);
    bye_func();
}
int main(int argc, char **argv)
{
    char name[1024];
    gets(name);
    srand(time(0));
    if (rand() % 2) hello(name, bye1);
    else hello(name, bye2);
}
```

Compile with: gcc -fno-stack-protector -z execstack -o hello hello.c

Run with: setarch x86_64 -R ./hello

Shellcode example

```
void bye1() { puts("Goodbye!"); }
void bye2() { puts("Farewell!"); }
void hello(char *name, void (*bye_func)())
{
    printf("Hello %s!\n", name);
    bye_func();
}
int main(int argc, char **argv)
{
    char name[1024];
    gets(name);
    srand(time(0));
    if (rand() % 2) hello(name, bye1);
    else hello(name, bye2);
}
```

1. Overflow gets
2. Overwrite return address of main
3. return to stack which contains the shellcode you wrote.

Compile with: `gcc -fno-stack-protector -z execstack -o hello hello.c`

Run with: `setarch x86_64 -R ./hello`

Stack-based buffer overflow caused by `gets`

Beyond control flow hijack

- A vulnerability in a program generally has more effects than just hijacking control flow.
- What other parts of the state do you gain control over?

```
#include <seccomp.h>
#include <assert.h>
int main(int argc, char **argv) {
    struct {
char shellcode[128];
int sandboxed;
    } program;
    program.sandboxed = 1;
    gets(program.shellcode);

    if (program.sandboxed) {
        scmp_filter_ctx ctx;
        ctx = seccomp_init(SCMP_ACT_KILL);
        assert(seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0) == 0);
        assert(seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0) == 0);
        assert(seccomp_load(ctx) == 0);
        ((void(*)())program.shellcode)();
    }
}
```

- If you only focus on hijacking to shellcode, you'll end up stuck in the jail.
- You can fill with null bytes instead.

Why You Are "Stuck in the Jail"

If you inject shellcode and hijack control to it:

- Your shellcode **likely tries to make a system call** (e.g., `execve`, `open`, etc.).

But the seccomp filter only allows read and write. Any other syscall (like `execve` or `open`) will be blocked.

- Your shellcode **will be killed by the sandbox.**
- You bypassed control flow, but the program is still sandboxed.
- You cannot make useful system calls, meaning you cannot escape the restricted environment.

but you may have fatal side-effects

- A vulnerability in a program generally has more effects than just hijacking control flow.

```
int main(int argc, char **argv) {  
    struct {  
        char shellcode[128];  
        char *shellcode_copy;  
    } program;  
    data.shellcode_copy = malloc(128);  
    gets(data.shellcode);  
    strcpy(data.shellcode_copy, data.shellcode);  
}
```

- If you corrupt the `shellcode_copy` pointer, the program will crash at `strcpy`!
- Keep state of the program viable (rewrite with a valid address, fix canary)

Just in Time Compilation

Existing Defenses

- Security Mitigations are Everywhere...
 - Data Execution Prevention prevents executing injected shellcode.
 - Stack Canaries protect against stack buffer overflow exploitation.
 - Address Space Layout Randomization makes control flow hijacking unreliable.
 - Modern exploitation is hard.

Just in Time Compilation

- Modern higher-level languages (JavaScript, Java, Lua, Python, etc) utilize Just in Time Compilation.
 1. Code is written in JavaScript.
 2. At runtime, necessary code is compiled to binary code.
 3. Binary code executes very fast.
- Of course, this means that an attacker that can inject higher-level code can influence the resulting native code **including immediate values stored!**

JIT Spraying Attack Technique

- Consider the following JavaScript:

```
var asdf = 0x050f3cb0c031;
```

- This might JIT to:

```
mov rdx, 0x050f3cb0c031
```

- Which assembles to:

```
48 ba 31 c0 b0 3c 0f 05 00 00
```

- What if we jump to the 0x31 via another vulnerability?

```
31 c0: xor eax, eax
```

```
b0 3c: mov al, 0x3c
```

```
0f 05: syscall
```

- Shellcode execution (in this case, exit())!

Shellcode vs JIT Spraying

Classic Shellcode	JIT Spraying
Injects binary machine code directly	Injects high-level code that generates machine code
Requires executable stack or heap	Uses JIT-generated executable memory
Requires memory control (e.g., gets())	Requires code injection in high-level languages like JavaScript
Often limited to native binaries	Works in sandboxed environments like browsers

But what about ASLR?

- Not a full mitigation, but....

“Spray and pray” – Repeatedly allocating sprayed JIT code increases the chance that one of the copies lands in a predictable location.

[illegible]

Reduces the effectiveness of ASLR - Attacker can guess an address.

Return Oriented Programming

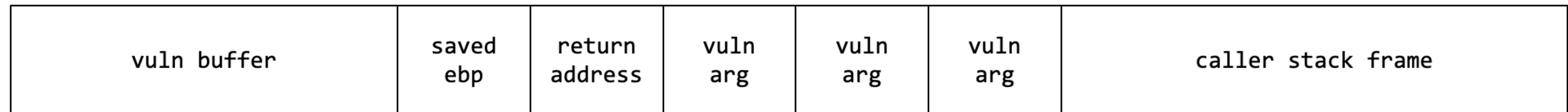
The "No-eXecute" bit

- Data Execution Prevention
- Modern architectures support memory permissions:
 - **PROT_READ** allows the process to read memory
 - **PROT_WRITE** allows the process to write memory
 - **PROT_EXEC** allows the process to execute memory
- Intuition: normally, all code is located in .text segments of the loaded ELF files. There is no need to execute code located on the stack or in the heap.
- By default in modern systems, the stack and the heap are not executable.

In the absence of Code Injection,
we turn to Code Reuse.

Return-to-libc (C standard Library)

- How can we deal with a non-executable stack?
- In the old times (32-bit x86), arguments were passed on the stack. During a stack-based buffer overflow, we could overwrite the return address *and* the arguments.



Return-to-libc



Alexander Peslyak

- How can we deal with a non-executable stack?
- In the old times (32-bit x86), arguments were passed on the stack. During a stack-based buffer overflow, we could overwrite the return address *and* the arguments.

AAAAAAAAAAAAAAAAAAAAAAAAA	saved ebp	system() address	fake return addr	address of "/bin/sh"	AAAAAA	AAAAAAAAAAAAAAAAAAAAAAAAA
---------------------------	--------------	---------------------	------------------------	-------------------------	--------	---------------------------

- When vuln() returns, it will call system("/bin/sh").

Not really applicable in AMD64

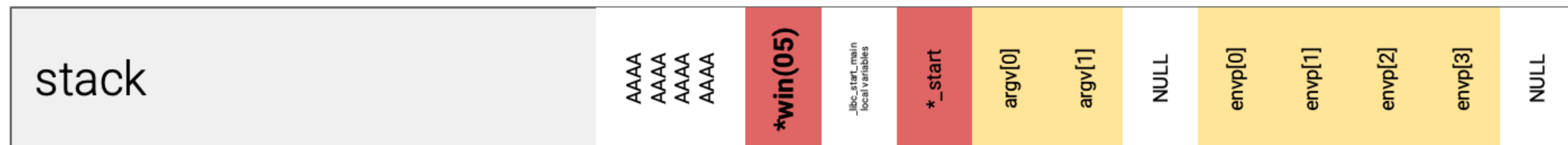
- Modern architectures don't take arguments on the stack.

- What can we do?

```
01 int main() {  
02     char name[16];  
03     read(0, name, 128);  
04 }  
05 int win() {  
06     sendfile(1, open("/flag", 0), 0, 1024);  
07 }
```

No arguments needed for win().
Control flow hijacking is
sufficient.

- We can jump to functions in the code!

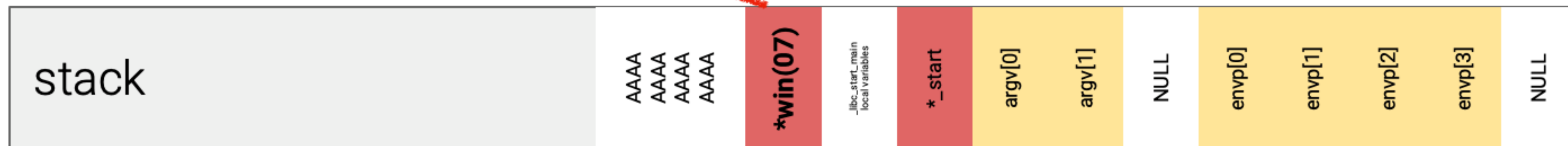


Not really applicable in AMD64

- Modern architectures don't take arguments on the stack.
- What can we do?

```
01 int main() {  
02     char name[16];  
03     read(0, name, 128);  
04 }  
05 int win(int tricky) {  
06     if (tricky != 1337) return;  
07     sendfile(1, open("/flag", 0), 0, 1024);  
08 }
```

- We can jump into the middle of functions in the code!



Not really applicable in AMD64

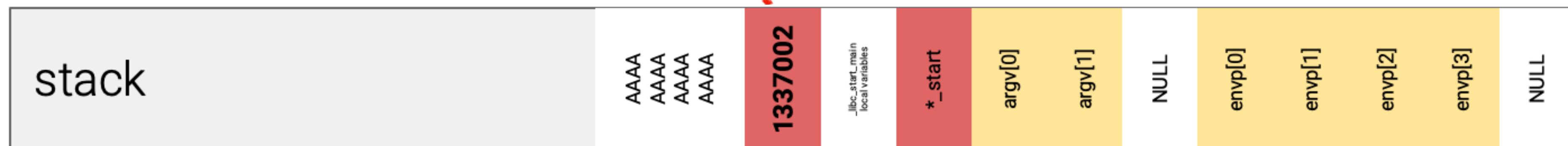
- Jitspraying

0x1337000 49 bc 31 c0 b0 3c 0f 05 90 90 mov r15, 0x9090050f3cb0c031

- If you jump to 0x1337002, you will execute:

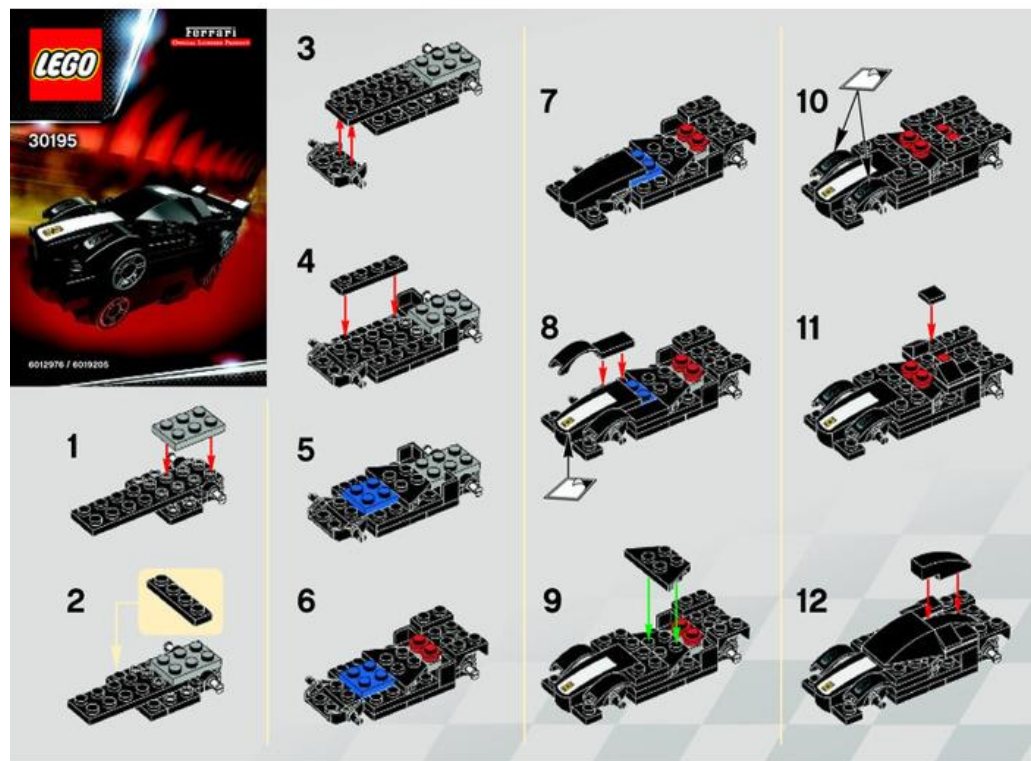
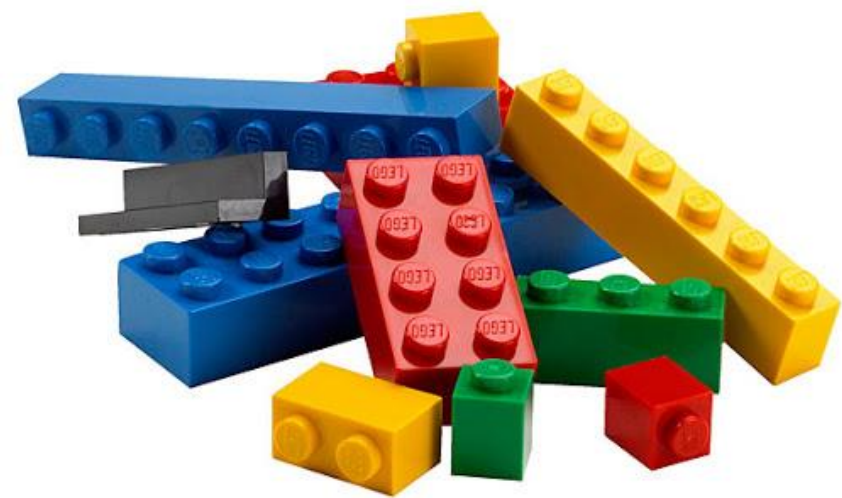
0x1337002	31 c0	xor eax, eax
0x1337003	b0 3c	mov al, 60
0x1337004	0f 05	syscall
0x1337005	90	nop
0x1337006	90	nop

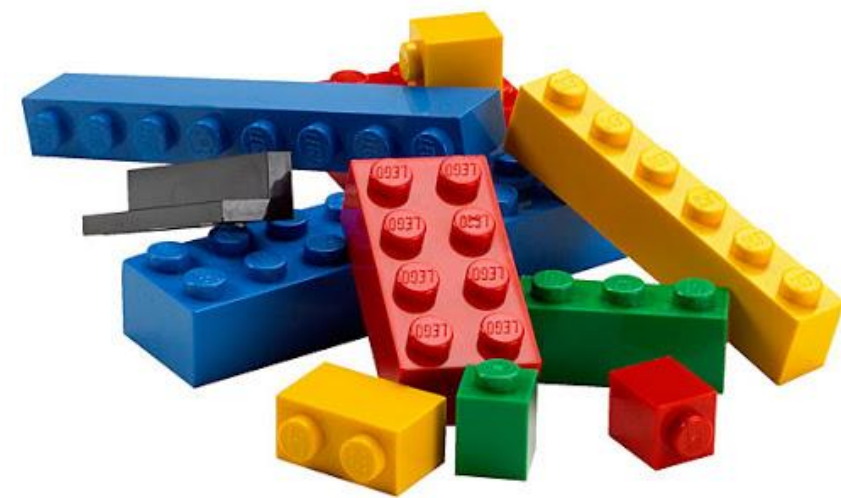
- We can jump into the middle of functions in the code!

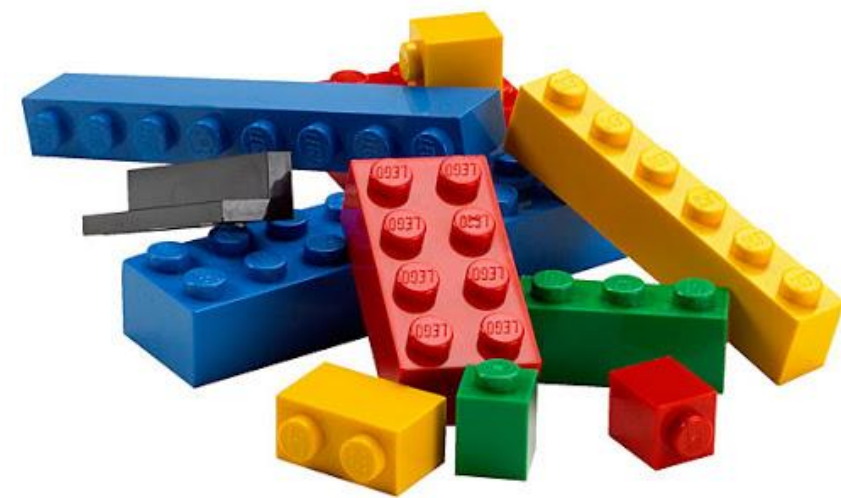


ROP

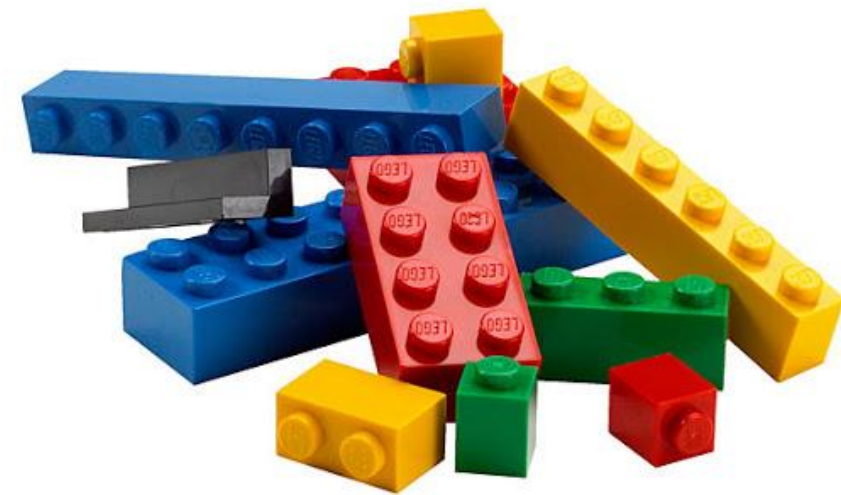
- The generalization of Return-to-libc is Return Oriented Programming.
- Chain returns one after the other
- These capabilities, when an attacker is able to overwrite return addresses on the stack, are extremely powerful.







Bytes in
.text section



The Stack, with
stored return
addresses

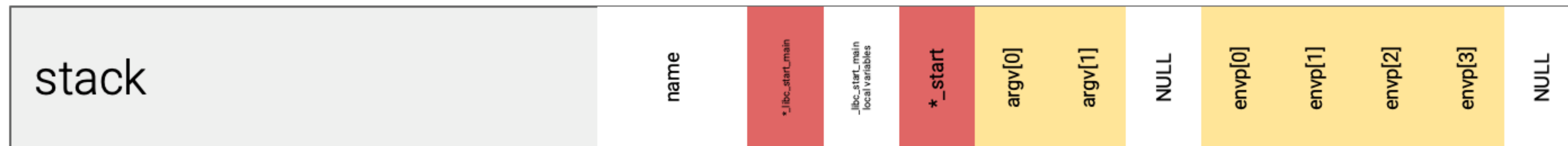


Actions taken by
the program



Conceptual Analogy

```
01 int main() {  
02     char name[16];  
03     read(0, name, 128);  
04 }  
05 int foo() {  
06     return open("/flag", 0);  
07 }  
08 int bar() {  
09     int x = open("/notflag", 0);  
10     sendfile(1, x, 0, 1024);  
11 }
```

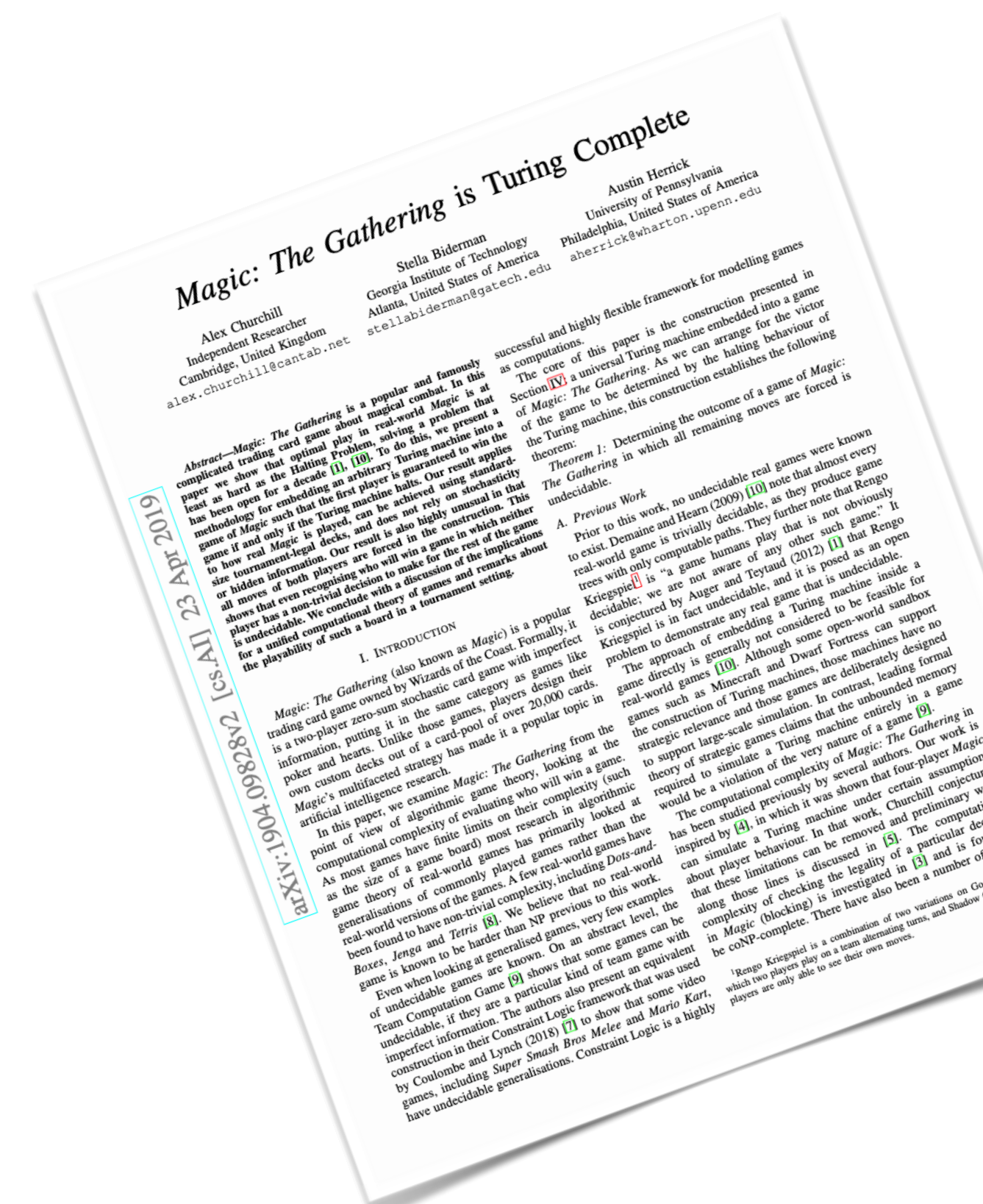


ROP by induction

- **Step 0:** overflow the stack
- **Step n:** by controlling the return address, you trigger a gadget:
 - `0x004005f3: pop rdi ; ret`
 - `ret`: Pops the next value from the stack into the instruction pointer (rip)
- **Step n+1:** when the gadget returns, it returns to an address you control (i.e., the next gadget)
- By chaining these gadgets, you can perform arbitrary actions in a ropchain!

ROP is basically shellcode

- A ROP gadget is equivalent to shellcode, but the instructions available to you are **WEIRD!**
- Hacker term: Programming the **Weird Machine** (https://en.wikipedia.org/wiki/Weird_machine), coined in 2009 by Sergey Bratus.
- Accidental turing completeness (http://beza1e1.tuxen.de/articles/accidentally_turing_complete.html).
- Fundamentally:
 - You get to choose from a set of bizarre meta-instructions already in memory.
 - You chain instructions using ret (and addresses on the stack).
 - Same lego pieces, new result!



ROP Concept: resources on the stack

	gadget 1 address	address of "/flag" (2 words to the right)	gadget 3 address	"/flag\0\0\0"	gadget 4 address	gadget 5 address	"/bin/cat"	gadget 6 address	gadget 7 address (syscall)	
--	---------------------	--	---------------------	---------------	---------------------	---------------------	------------	---------------------	----------------------------------	--

- If you know where your ropchain is on the stack, you can include resources (such as a "/flag" string) in your chain.
- But how do we prevent "/flag" and "/bin/cat" from being interpreted as gadget addresses (obviously, that would crash the program)?

ROP Concept: janitorial gadgets

- Some of your gadgets will be there simply to unbreak your ropchain, and that is okay!
- Example: stack fix-up gadgets!

```
pop r12; pop rdi; pop rsi; ret  
add rsp, 0x40; ret
```



```
rsp -> r12  
rsp+8 -> rdi  
rsp+16 -> rsi  
Then ret uses the next value on the  
stack as the next instruction  
address.
```
- This lets you skip data on your stack.

ROP Concept: storing values into registers

	gadget 1 address (0x400400)	desired rax value	gadget 2 address	"/flag"	gadget 3 address	gadget 4 address	"/bin/cat"	gadget 5 address	gadget 6 address (syscall)	
--	-----------------------------------	----------------------	---------------------	---------	---------------------	---------------------	------------	---------------------	----------------------------------	--

- You can store values into registers using register-popping gadgets. For example:

`0x400400: pop rax; ret`

- This gadget will pop rax off the stack, then return to gadget 2. Now you've set rax!

ROP Concept: rare and common gadgets

- Some gadgets are rarer than others. Relatively common ones:
 - `ret` (at the end of every function)
 - `leave; ret` (at the end of many functions)
 - `pop REG; ret` (restoring callee-saved registers before returning)
 - `mov rax, REG; ret` (setting the return value before returning)
- Because you can jump into the middle of an instruction, instructions don't have to be common to appear in common gadgets!
- Example: every `add rsp, 0x08; ret` also contains a `add esp, 0x08;` if you jump past the REX ("H") prefix.

ROP Concept: storing addresses into registers

- `lea` gadgets that do exactly what you want are rare (long instruction, and mostly used in the beginning or middle of functions, not near a `ret`).
 - Alternative #1: `push rsp; pop rax; ret` (equivalent to `mov rax, rsp`) will get the stack address into `rax`.
 - Alternative #2: `add rax, rsp; ret` (put 0 in `rax` using another gadget, then `add rsp`. not perfect, but will conceptually get `rsp` into `rax`)
 - Alternative #3: `xchg rax, rsp; ret` (swap `rax` and `rsp`. DANGEROUS)
- Once you have the stack address, later gadgets can dynamically compute necessary addresses on the stack instead of having them hardcoded. Now you (might not) need a stack leak!

ROP Concept: stack pivot

- Don't like your stack? Too limiting? Try a stack pivot!

```
xchg rax, rsp; ret  
pop rsp; ...; ret
```

- This will pivot your stack to point elsewhere. That `ret` will read the return address off of your new stack!

ROP Concept: data transfer

- Shellcode needs to move data around. So do ropchains. One common gadget:

```
add byte [rcx], al ; pop rbp ; ret
```

- Obviously, this would require a gadget to set rcx (surprisingly rare) and rax (less rare).

ROP Concept: syscalls are rare

- In shellcode, you use syscall to invoke system calls. This instruction is quite rare in normal programs (even as a part of other instructions).
- You might have to call library functions, instead!
- Keep it Simple.

ROP Concept: Know Your Environment

- Your ropchain doesn't run in a vacuum! When it starts, there are useful addresses all over the place.
 - code, stack, heap addresses in registers
 - code, stack, heap addresses all over the stack

ROP Concept: finding the rop gadgets

- Many tools available (<https://github.com/zardus/ctf-tools> has installers for 3!). For example, rp++:

```
# rp++ --unique -r2 -f /bin/bash
```

- Can also try greater values than 2, but long gadgets become increasingly unstable (side-effects!).
- From here, regular expressions are your friends.

```
# rp++ --unique -r2 -f /bin/bash | grep -P "(add|sub|mov) rax, r.."
```

Complications

Issue 1: Limited Control

- Sometimes, your control of the stack is limited.
 - Limited overflow size.
 - Inability to input NULL bytes. Often, we can still trigger one gadget!
`many gadget addresses may contain a null byte`
`(it is just 0x00 in hexadecimal)`
- Can we own the system with one gadget?

The Magic Gadget

- Yes we can!
- Consider `system()` ...
 - it has to set up a call to `execve("/bin/sh", {"/bin/sh", "-c", command}, env)` ;
 - what if we jump partway through?
- This actually works! If you get lucky with register values and stack setup, you can often trigger `/bin/sh` by jumping partway into `system()`. This location is called the magic gadget.
- More useful for you: trigger `execve(some_file)` ; and create a `some_file` file that reads the flag.

Issue 2: Address Space Layout Randomization

- **Workaround 1:** Often, you can restart the whole program by jumping back to main or some other high-level function.
 - have another shot at the exploit, but with potentially more data.
 - If you're returning from main (into libc), you might be able to hit the magic gadget!
- **Workaround 2:** Situation information disclosure. Leak addresses during the exploit so you can adjust your ROP chain accordingly.

Issue 3: Stack Canaries

- Need to either leak or bypass in some other way.

Issue 4: Exotic Academic Solutions

- ROP is part of a long-term cat and mouse game between attackers and defenders.
- Anti-ROP approaches:
 - removing ROP gadgets (too onerous):
 - G-Free: Defeating Return-Oriented Programming through Gadget-less Binaries
 - detecting ROP in progress (deployed, but bypassable):
 - kBouncer: Efficient and Transparent ROP Mitigation
 - ROPecker: A Generic and Practical Approach for Defending Against ROP Attacks
 - control flow integrity

Control Flow Integrity

- Core idea: whenever a hijackable control flow transfer occurs, make sure its target is something it's supposed to be able to return to!
- This spawned a whole arms race.
- Counter-CFI techniques:
 - B(lock)OP: ROP on a block (or multi-block) level by carefully compensating for side-effects.
 - J(ump)OP: instead of returns, use indirect jumps to control execution flow
 - C(all)OP: instead of returns, use indirect calls to control execution flow
 - S(ignreturn)ROP: instead of returns, use the signreturn system call
 - D(ata)OP: instead of hijacking control flow, carefully overwrite the program's data to puppet it

Control Flow Integrity: Intel Edition!

- Intel recently (late 2020) released processors with Control-flow Enforcement Technology (CET).
- Among other things, CET adds the endbr64 instruction.
- On CET-enabled CPUs, indirect jumps (including ret, jmp rax, call rdx, etc) MUST end up at an endbr64 instruction or the program will terminate.
- This is still bypassable by some advanced ROP techniques (Block Oriented Programming, SROP, etc), but it will significantly complicate exploitation.

Issue: Hacking Blind?

- Can you exploit a program that you do not have? In certain situations, yes!
- Intuition: things are only randomized at program start time.
- The standard blind attack requires a forking service.
 - Break ASLR and the canary byte-by-byte. Now we can redirect memory semi-controllably.
 - Redirect memory until we have a survival signal (i.e., an address that doesn't crash).
 - Use the survival signal to find non-crashing ROP gadgets.
 - Find functionality to produce output.
 - Leak the program.
 - Hack it.